DSA STUDY BLUEPRINT SERIES

99. Recover BST

Correcting Swapped Nodes in BSTs

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Two BST nodes are swapped. Correct the tree layout.
- Inorder values will show descending anomalies. Track them using pointers.

Memory & Execution Blueprint

Inorder: [1, 5, 3, 4, 2] ⇒ Swapped pairs are 5 and 2 ⇒ Swap back to correct BST.

C++ Implementation Reference

Standard code layout and syntax

```
TreeNode *first = nullptr, *second = nullptr, *prevNode = nullptr;
void detectSwapped(TreeNode* root) {
    if (!root) return;
    detectSwapped(root->left);
    if (prevNode && root->val < prevNode->val) {
        if (!first) first = prevNode;
        second = root;
    }
    prevNode = root;
    detectSwapped(root->right);
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N)
Space Complexity	O(H)

Key Practices & Gotchas

- **Important:** Swap the node values (`swap(first->val, second->val)`) after search traversal completes.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

Active Node	Valid range limit	Comparison check	Recursive branch step
15 (Root)	[-∞, +∞]	True (-∞ < 15 < +∞)	Recurse Left child with range [-∞, 15]
10 (Left)	[-∞, 15]	True (-∞ < 10 < 15)	Recurse Right child with range [10, 15]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.